Совет нейросетей

Подробное описание логики работы приложения
для независимого анализа

Назначение: несколько языковых моделей разных разработчиков независимо отвечают на вопрос,
анонимно критикуют друг друга, дорабатывают ответы, голосуют, а председатель пишет итоговое решение.

Стек: Python 3.12+, asyncio, httpx, pydantic, FastAPI, WeasyPrint, python-docx; модели через OpenRouter.
Интерфейсы: командная строка, веб-интерфейс, Telegram-бот.
Исходный код: папка `ai-council/`, ветка `claude/multi-ai-comparison-tool-50nc5z`.
Документ составлен по коду на 10.11.2026.

Содержание

1. Идея и принципы ... 3
2. Архитектура ... 4
3. Участники, роли и настройки ... 6
4. Заседание: этапы по шагам ... 8
   - 4.0 Подготовка: вложения, уточнения, описание картинок ... 8
   - 4.1 Постановка задачи и критерии ... 8
   - 4.2 Независимые предложения и анонимизация ... 8
   - 4.3 Критика ... 9
   - 4.4 Доработка ... 9
   - 4.5 Голосование ... 9
   - 4.6 Итоговое решение председателя ... 10
5. Режимы заседания и число обращений к моделям ... 11
6. Подсчёт голосов (формулы и пример) ... 12
7. Надёжность: повторы, проверка ответов, кворум, бюджет ... 14
8. Вложения: документы и изображения ... 16
9. Уточняющие вопросы и повтор заседания ... 17
10. Голосовой ввод ... 18
11. Предварительная оценка стоимости ... 19
12. Хранение: архив, стенограмма, отчёты ... 20
13. Интерфейсы: CLI, веб, Telegram ... 21
14. Статистика ... 22
15. Безопасность и развёртывание ... 23
16. Известные ограничения и вопросы для анализа ... 24

Приложение. Карта исходного кода ... 26

1. Идея и принципы

Приложение организует «совет» из нескольких больших языковых моделей (Claude, ChatGPT, Gemini, Kimi, DeepSeek и др.). Вместо одного ответа одной модели пользователь получает решение, прошедшее через независимые предложения, перекрёстную критику, доработку и голосование. Цель — снизить влияние ошибок и «слепых пятен» отдельной модели и получить обоснованный выбор с видимыми альтернативами и рисками.

Ключевые принципы

- **Независимость.** На этапе предложений каждая модель отвечает вслепую — не видя ответов других.
- **Анонимность.** Предложения перемешиваются случайно и получают метки A, B, C… Модели критикуют и голосуют за метки, а не за авторов, чтобы оценивалась идея, а не «бренд» модели.
- **Запрет голоса за себя.** Модель не оценивает и не ранжирует собственный вариант.
- **Измеримость.** Оценки по критериям (1–10) и ранжирование сводятся в формальный подсчёт (нормированный метод Борда).
- **Итог от председателя.** Одна назначенная модель пишет итоговый ответ, опираясь на победителя и на лучшие идеи других вариантов. При расхождении с голосованием председатель обязан объяснить причину.
- **Строгий формат.** Каждый ответ модели — JSON по заданной схеме; ответ проверяется, при ошибке модель просят исправиться.
- **Прозрачность.** Сохраняется полная стенограмма: каждый запрос, сырой ответ, попытки, стоимость, токены.
- **Контроль расходов.** Жёсткий лимит стоимости одного заседания и предварительная оценка цены до запуска.

Что получает пользователь

- Итоговое решение (готовый к использованию ответ в формате Markdown), обоснование выбора, особые мнения, риски.
- Таблицу голосования, все предложения (исходные и доработанные), отзывы критиков, бюллетени.
- Отчёты в PDF, Word и Markdown — полный, итог (решение с обоснованием) или только решение совета.
- Расходы по каждой модели, длительность, сведения о сбоях.

2. Архитектура

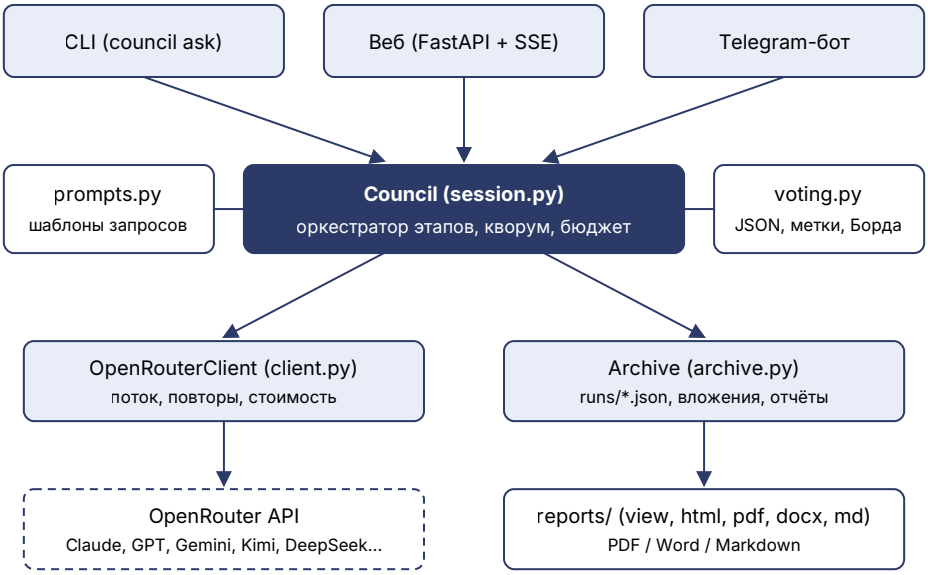


Рис. 1. Основные компоненты и направления вызовов

Слои

Слой	Ответственность
Интерфейсы (`cli.py`, `web/app.py`, `telegram/bot.py`)	Приём вопроса и настроек, загрузка вложений, показ хода заседания в реальном времени, выдача результата и отчётов, архив.
Оркестратор (`session.py`, класс `Council`)	Последовательность этапов, параллельные вызовы, анонимизация, проверка ответов, кворум, бюджет, стенограмма, события прогресса.
Промпты (`prompts.py`)	Тексты запросов к моделям на каждом этапе (на английском — модели точнее следуют инструкциям; ответы — на языке из настроек).
Схемы (`schemas.py`)	Строгие pydantic-модели ответов: Framing, Proposal, Revision, CritiqueSet, Ballot, Verdict, ImageNotes.
Голосование (`voting.py`)	Извлечение JSON из ответа, нормализация меток и оценок, назначение анонимных меток, подсчёт итогов.
Клиент (`client.py`)	HTTP-запросы к OpenRouter с потоковой передачей, сетевые повторы, учёт стоимости, обработка пустых и обрезанных ответов.
Хранилище (`archive.py`, `report.py`)	Сохранение стенограммы, файлов-вложений и отчётов; поиск; пересборка отчётов; цепочки уточнений.
Отчёты (`reports/`)	Единое представление заседания (`View`) и генерация HTML/PDF, DOCX, Markdown; фирменное оформление.
Вспомогательное	`attachments.py` (файлы), `transcribe.py` (голос), `estimate.py` (цена, модели, зрение), `stats.py` (статистика), `mock.py` (демо-режим без затрат), `config.py` (настройки).

Поток данных одного заседания

1. Интерфейс собирает вопрос, режим, участников, роли, бюджет, вложения и (если это уточнение) ссылку на предыдущее заседание.
2. Строится конфигурация: базовый `config.yaml` + пользовательские переопределения (`with_overrides`), с проверкой корректности.
3. Загружается каталог моделей OpenRouter (кэш 10 минут) — для проверки, какие модели «видят» картинки.
4. Создаётся `Council` с клиентом (реальным или демо) и обработчиком событий; вызывается `run()`.
5. Каждый этап порождает события (`stage`, `call`, `member_done`, `member_failed`, `cost`, `tally`, `verdict`…), которые интерфейс показывает пользователю.
6. Результат (`SessionResult`) сохраняется в архив как JSON, вложения — в отдельную папку, затем строятся отчёты.

3. Участники, роли и настройки

Состав совета по умолчанию

Имя	Модель (OpenRouter)	Включена	Роль
claude	`anthropic/claude-sonnet-4.5`	да	председатель
chatgpt	`openai/gpt-5`	да	—
gemini	`google/gemini-2.5-pro`	да	—
kimi	`moonshotai/kimi-k2`	да	—
deepseek	`deepseek/deepseek-chat`	да	адвокат дьявола
grok, qwen, mistral, llama, perplexity	`x-ai/grok-4`, `qwen/qwen3-max`, `mistralai/mistral-large`, `meta-llama/llama-4-maverick`, `perplexity/sonar-pro`	нет	резерв

Роли

- **Участник.** Предлагает вариант, критикует чужие, дорабатывает свой, голосует.
- **Председатель** (`chair`). Дополнительно формулирует уточнённый вопрос и критерии (этап 0) и пишет итоговое решение (этап 6). Является одновременно обычным участником со своим вариантом; на этапах 0 и 6 получает инструкцию быть нейтральным, «включая варианты, которые могут быть его собственными».
- **Адвокат дьявола** (`devil_advocate`, необязательно). На этапах предложений, критики и доработки получает дополнительную инструкцию искать неочевидные альтернативы, скрытые допущения и слабые места, оставаясь конструктивным.

Системная инструкция (общая для всех)

Каждая модель получает роль «члена экспертного совета из моделей разных лабораторий» и правила: думать независимо, не соглашаться только потому, что согласны другие; быть конкретной; критиковать честно, без лести; оценивать идеи, а не авторов; писать человекочитаемый текст на заданном языке; отвечать **ровно одним JSON-объектом** без обрамления.

Основные настройки (`config.yaml`)

Параметр	По умолчанию	Смысл
`mode`	debate	quick / debate / evaluate (см. раздел 5)
`debate_rounds`	1	число раундов «критика → доработка» (0–5)
`min_quorum`	2	минимум полученных предложений, иначе заседание прерывается
`max_cost_usd`	2.0	жёсткий лимит стоимости одного заседания
`temperature`	0.7	температура генерации (можно задать для каждого участника)

`max_tokens` / `max_tokens_cap`	8000 / 32000	начальный лимит ответа и потолок, до которого он удваивается при обрезке
`reasoning_effort`	low	глубина скрытых рассуждений «думающих» моделей: none/low/medium/high
`timeout_s`	180	таймаут ожидания; весь поток ответа ограничен ×3 (540 с)
`max_retries`	3	повторы при сетевых ошибках, 429, 5xx, пустом ответе
`json_retries`	2	повторы, если ответ не соответствует схеме
`max_concurrency`	8	максимум одновременных запросов к моделям
`language`	Russian	язык ответов
`transcribe_model`	google/ gemini-2.5-flash	модель расшифровки голосовых сообщений
`branding`	пусто	название, логотип, цвет и колонтитул отчётов

При загрузке проверяется: имена участников уникальны; председатель и адвокат дьявола входят во включённых участников; число включённых не меньше кворума. Пользователь в вебе и Telegram может переопределить состав, председателя, адвоката и бюджет для одного заседания — результат проверяется по тем же правилам; если председатель выключен, им становится первый включённый.

4. Заседание: этапы по шагам



Рис. 2. Последовательность этапов (голубые — все участники параллельно)

4.0 Подготовка: вложения, уточнения, описание картинок

- Если вопрос — уточнение к прошлому заседанию, в оркестратор передаётся история (до 4 предыдущих пар «вопрос — решение»), а вложения исходного заседания объединяются с новыми (без дублей). Подробнее — раздел 9.
- Если есть изображения, а среди участников есть модели без поддержки картинок, одна «зрячая» модель (предпочтительно председатель) заранее описывает каждое изображение текстом: весь видимый текст дословно, числа, содержимое таблиц, оси и значения графиков, элементы схем. Это отдельный вызов (этап `describe`). Описание затем получают «незрячие» модели вместо самих картинок. Если ни одна модель не видит картинки — пользователь получает предупреждение, изображения пропускаются. Подробнее — раздел 8.

4.1 Постановка задачи и критерии (этап 0, только председатель)

Если пользователь сам не задал критерии, председатель получает вопрос (и дополнительный контекст) и должен: (1) переформулировать вопрос точно и нейтрально, сохранив намерение пользователя и не отвечая на него; (2) предложить 3–5 коротких критериев оценки (1–4 слова каждый). Схема ответа: `{"refined_question", "criteria"}`, критериев 1–7.

Если председатель не ответил или ответ некорректен, заседание продолжается с исходной формулировкой и критериями по умолчанию: «Эффективность», «Реалистичность», «Риски». Если критерии заданы пользователем, этап пропускается.

Дальше во всех запросах используется **уточнённая** формулировка вопроса.

4.2 Независимые предложения и анонимизация (этапы 1–2)

Все включённые участники параллельно получают вопрос, контекст и критерии и предлагают **свой лучший вариант**, не видя чужих. Схема ответа:

```
{"title": "краткое название", "summary": "сам вариант, полно и конкретно (Markdown)",
 "reasoning": "почему это лучший выбор", "pros": [...], "risks": [...], "confidence": 0.0–1.0}
```

Участник, не ответивший после всех повторов, выбывает из этапа (фиксируется сбой). Если получено меньше `min_quorum` предложений — заседание прерывается с ошибкой «нет кворума».

Анонимизация: список ответивших участников случайно перемешивается (генератор случайных чисел можно зафиксировать параметром `seed`) и им выдаются метки A, B, C…

Таблица «метка → автор» хранится только в оркестраторе и раскрывается пользователю в результатах, но не моделям.

В режиме «Оценка вариантов» этап 1 пропускается: варианты задаёт пользователь (от 2 до 26), им присваиваются метки по порядку, автор у них отсутствует, название — первая строка варианта (до 80 символов).

4.3 Критика (этап 3)

Каждый участник получает все варианты, **кроме своего**, и для каждого ставит оценку 1–10 по каждому критерию, перечисляет сильные и слабые стороны. Схема:

```
{"reviews": [{"label": "A", "scores": {"Критерий 1": 7, ...},
              "strengths": [...], "weaknesses": [...]}, ...]}
```

Ответ проходит дополнительную проверку (`normalize_critique`):

- метки приводятся к виду «A» (убираются слова «Proposal», «Вариант», скобки, кавычки), чужие метки отбрасываются;
- названия критериев сопоставляются без учёта регистра; если не совпадают, но число оценок равно числу критериев — сопоставляются по порядку;
- каждая оценка должна быть в диапазоне 1–10;
- должны быть оценены **все** ожидаемые варианты, иначе ответ считается некорректным и модель просят исправить его.

4.4 Доработка (этап 4, только режим «Дебаты»)

Каждый автор получает: свой текущий вариант; **анонимные** отзывы других участников именно на свой вариант (оценки, сильные и слабые стороны, без имён рецензентов); краткое содержание чужих вариантов (название + первые 400 символов). Инструкция: исправить реальные слабости, сохранить сильное, заимствовать хорошие идеи; если чужой вариант явно лучше — можно перестроить свой на его основе; не менять ничего «в угоду критикам». К схеме предложения добавляется поле `changes` — что изменено и почему.

Доработанный вариант заменяет исходный под той же меткой. Если автор не смог доработать — остаётся прежняя версия. Исходные версии сохраняются отдельно (`initial_proposals`) для отчёта.

При `debate_rounds` > 1 цикл «критика → доработка» повторяется. Для подсчёта и для председателя используются отзывы **последнего** раунда критики.

4.5 Голосование (этап 5)

Каждый участник получает финальные версии всех вариантов, кроме своего, и ранжирует их от лучшего к худшему с кратким обоснованием: `{"ranking": ["B", "D", "A", ...], "rationale": "..."}`. Проверка (`normalize_ranking`): метки нормализуются, повторы и посторонние метки удаляются; если в ранжировании не хватает какого-либо ожидаемого варианта — ответ возвращается модели на исправление.

Затем выполняется подсчёт (раздел 6). Результат — упорядоченная таблица: метка, балл Борда, средняя оценка критики, число первых мест, число бюллетеней.

4.6 Итоговое решение председателя (этап 6)

Председатель получает: уточнённый вопрос, критерии, все финальные варианты, текст таблицы голосования и сводку критики (по каждому варианту до 2 сильных и 2 слабых сторон от каждого рецензента, не более 12 пунктов на вариант). Он должен вернуть:

```
{"winner": "метка победителя",
 "final_answer": "полный готовый ответ пользователю на основе победителя,
                  улучшенный лучшими идеями других вариантов (Markdown)",
 "why": "почему победил этот вариант",
 "dissent": ["заметные особые мнения и сильные альтернативы"],
 "risks": ["основные риски и как их снизить"]}
```

Голосование — «сильный аргумент», но председатель может выбрать иначе, обязательно обосновав это в `why`. Метка победителя проверяется: она должна быть одной из существующих.

Запасной путь: если председатель не ответил или ответ некорректен после всех повторов, победителем становится лидер голосования, итоговым ответом — его текст без синтеза, с пометкой «Председатель не ответил; победитель определён по голосованию».

5. Режимы заседания и число обращений к моделям

Режим	Предложения	Критика	Доработка	Назначение
Быстрый (quick)	модели	нет	нет	Дешёвый ответ: предложения → голосование → итог. Средняя оценка критики в таблице отсутствует.
Дебаты (debate)	модели	N раундов	N раундов	Основной режим: полный цикл с доработкой (по умолчанию N = 1).
Оценка вариантов (evaluate)	пользователь	1 раунд	нет	Сравнение готовых вариантов пользователя (2–26). Каждая модель оценивает и ранжирует все варианты. Включается автоматически, если переданы варианты.

Число вызовов моделей

Обозначим M — число участников, N — число раундов. Без учёта повторов:

Этап	Быстрый	Дебаты	Оценка
Описание картинок (если нужно)	1	1	1
Постановка (если критерии не заданы)	1	1	1
Предложения	M	M	0
Критика	0	M·N	M
Доработка	0	M·N	0
Голосование	M	M	M
Итог	1	1	1
Итого при M = 5, N = 1	**12**	**22**	**12**

Внутри этапа запросы идут параллельно (не более `max_concurrency` одновременно), этапы — строго последовательно, потому что каждый следующий зависит от результатов предыдущего. Поэтому длительность заседания определяется самой медленной моделью на каждом этапе.

6. Подсчёт голосов (формулы и пример)

Нормированный балл Борда

Каждый бюллетень — ранжирование n вариантов (без собственного). Позиция i (с нуля) даёт варианту

```
points = (n − 1 − i) / (n − 1)      — первое место = 1, последнее = 0;
                                       если в бюллетене один вариант, он получает 1.
borda(вариант) = среднее points по всем бюллетеням, где он встречается   (0..1)
```

Нормировка нужна потому, что бюллетени разной длины: участник не голосует за себя, а в режиме «Оценка» авторов нет — без нормировки варианты, получившие больше бюллетеней, имели бы преимущество.

Средняя оценка критики

```
avg_score(вариант) = среднее по рецензентам от (среднее оценок рецензента по всем
критериям)   (1..10)
```

Критерии равновесны. Используются отзывы последнего раунда критики.

Порядок и разрешение ничьих

1. по убыванию балла Борда;
2. при равенстве — по убыванию средней оценки критики;
3. при равенстве — по алфавиту метки.

Также считается число первых мест (информационно, на порядок не влияет). Лидер таблицы передаётся председателю как ориентир; окончательный победитель — тот, кого выбрал председатель (обычно совпадает).

Пример (4 участника)

Авторы: A — gemini, B — claude, C — deepseek, D — chatgpt. Каждый ранжирует три чужих варианта (n = 3, баллы 1 / 0,5 / 0):

Голосует	1-е	2-е	3-е
gemini (A)	B	D	C
claude (B)	D	A	C
deepseek (C)	B	A	D
chatgpt (D)	B	A	C

Вариант	Баллы	Борда	Первых мест
B	1, 1, 1	1,00	3/3
A	0,5, 0,5, 0,5	0,50	0/3
D	0,5, 1, 0	0,50	1/3
C	0, 0, 0	0,00	0/3

A и D равны по Борда — их порядок решит средняя оценка критики. Председателю передаётся текст вида `1. B: borda=1.00, critique=8.1, first places=3/3`.

7. Надёжность: повторы, проверка ответов, кворум, бюджет

Каждое обращение к модели проходит два уровня защиты: сетевой (в клиенте) и смысловой (в оркестраторе).

Уровень 1. Клиент OpenRouter

- Запрос с потоковой передачей (SSE). Это позволяет видеть, что модель жива: «думает» (идут скрытые рассуждения или служебные строки сервера) или «пишет» (идёт текст ответа).
- В запросе передаются `usage.include` (OpenRouter возвращает фактическую стоимость) и `reasoning.effort` (глубина рассуждений для «думающих» моделей).
- **Временные ошибки** (HTTP 408, 409, 425, 429, 500, 502, 503, 504, сетевые сбои, не-JSON ответ, пустой список вариантов, поток дольше 3×таймаута) → повтор с экспоненциальной задержкой `min(2^попытка, 30) + случайная доля` секунд, до `max_retries + 1` попыток.
- **Постоянные ошибки** (прочие 4xx, ошибка провайдера) → немедленный отказ.
- **Пустой ответ**: если модель израсходовала лимит на скрытые рассуждения (`finish_reason = length`), лимит удваивается (до `max_tokens_cap`) и запрос повторяется сразу; иначе — обычный повтор.
- Стоимость и токены **всех** попыток суммируются, включая неудачные; при отказе потраченное передаётся вверх и учитывается.

Уровень 2. Оркестратор (`_ask`)

- Ответ разбирается терпимо (`extract_json`): снимается обёртка ```` ```json ````, отбрасывается текст до и после, допускаются переводы строк внутри строк и лишняя запятая перед `}` / `]`; берётся первый JSON-объект.
- Объект проверяется по схеме pydantic и дополнительными правилами этапа (метки, критерии, диапазоны, полнота).
- Если ответ **обрезан по лимиту** — тот же запрос повторяется с удвоенным лимитом (обрывок не пересылается).
- Если ответ **не по формату** — в диалог добавляются сам ответ и сообщение «ваш ответ не принят: <ошибка>, ответьте ещё раз одним корректным JSON» (модель видит свою ошибку).
- Всего до `json_retries + 1` попыток (по умолчанию 3). Каждая попытка записывается в стенограмму с сырым ответом и ошибкой.

Выбывание участников и кворум

- На параллельных этапах отказ одного участника не останавливает заседание: он выбывает из этого этапа, сбой записывается в статистику и показывается пользователю.
- Выбывший на этапе предложений не имеет своего варианта, но может участвовать в критике и голосовании.

- Кворум проверяется только после этапа предложений: получено меньше `min_quorum` → заседание прерывается.
- Отказ председателя на этапе 0 → критерии по умолчанию; на этапе 6 → итог по голосованию (раздел 4.6).

Бюджет

Перед **каждым** запросом сравнивается накопленная стоимость с `max_cost_usd`. При достижении лимита заседание прерывается полностью (исключение «лимит бюджета» не гасится на параллельных этапах). Так как запросы одного этапа уходят параллельно, фактическая стоимость может превысить лимит на стоимость запросов, уже отправленных в этот момент.

Отмена

Пользователь может остановить заседание в любом интерфейсе; задача отменяется, незавершённое заседание в архив не сохраняется (в вебе оно видно в списке как «остановлено» до перезапуска сервера).

События прогресса

Для каждого обращения генерируются фазы: в очереди → отправлено → думает → пишет → проверяю → готово / повтор / сбой, с прошедшим временем, числом символов, номером попытки и причиной повтора. Частые обновления «думает/пишет» прореживаются (не чаще раза в 0,8 с на участника), смена фазы отправляется сразу. Веб-страница дополнительно получает «сердцебиение» раз в 10 с.

8. Вложения: документы и изображения

Поддерживаемые файлы и лимиты

Тип	Обработка
Изображения: JPG, PNG, WEBP, GIF, BMP	Поворот по EXIF, уменьшение до 1600 px по длинной стороне, передача моделям как картинки.
PDF	Извлечение текста по страницам (pypdf).
Word (.docx)	Текст абзацев и таблиц.
Excel (.xlsx, .xlsm)	Листы как таблицы (до 2000 строк на лист).
Текстовые: TXT, MD, CSV, TSV, JSON, XML, YAML, HTML, код	Как текст; CSV/TSV — как таблица (до 3000 строк).

Лимиты: до 10 файлов, до 20 МБ на файл, до 60 000 символов текста на документ, до 150 000 символов на все документы запроса (лишнее обрезается с предупреждением).

Как вложения попадают к моделям

- Текст документов, примечание о картинках и (для уточнений) история добавляются **перед** текстом запроса на этапах: постановка, предложения, критика, доработка, итог. На этапе голосования вложения не передаются (экономия; модели уже знакомы с материалами).
- Модели, поддерживающие изображения, получают сами картинки. Поддержка определяется по каталогу OpenRouter (`input_modalities`); если каталог недоступен — по списку известных «незрячих» моделей (deepseek, kimi-k2, qwen3-max, mistral-large); если неизвестно — считается, что модель видит.
- «Незрячие» модели получают текстовые описания картинок, подготовленные на этапе 4.0.
- Файлы сохраняются в архиве вместе с заседанием; в отчётах выводится их список, картинки — миниатюрами.

9. Уточняющие вопросы и повтор заседания

Уточнение («Задать уточнение» / «Уточнить» / `--followup`)

1. Пользователь выбирает прошлое заседание и задаёт новый вопрос.
2. Из архива берётся цепочка: история самого прошлого заседания (если оно тоже было уточнением) плюс его пара «вопрос — итоговое решение» (решение обрезается до 4000 символов). Хранятся последние 4 звена.
3. Вложения прошлого заседания загружаются из архива и объединяются с новыми (дубли по имени и размеру отбрасываются, всего ≤ 10).
4. Новое заседание проходит **полный цикл**, но в начало запросов (этапы 0, 1, 3, 4, 6) добавляется блок: «Это УТОЧНЯЮЩИЙ вопрос. Ранее совет обсуждал тему (от старых к новым). Опирайтесь на эти выводы; пересматривайте их, если новый вопрос этого требует» и список прошлых вопросов и решений.
5. В результате сохраняются ссылка на родителя (`followup_of`) и цепочка (`thread`); в интерфейсе и отчётах показывается «↳ Уточнение к заседанию …».

Совет при уточнении видит только итоговые решения прошлых заседаний, а не их предложения, критику и голосование.

Повтор заседания

При каждом запуске сохраняются исходные настройки (`request`): вопрос, режим, раунды, критерии, варианты, контекст, участники, председатель, адвокат дьявола, бюджет. «Повторить» заполняет форму этими значениями и копирует вложения; пользователь может всё изменить перед запуском. Для старых заседаний без `request` настройки восстанавливаются из стенограммы. Повтор — это новое независимое заседание; результаты моделей могут отличаться (температура 0,7, случайные метки).

10. Голосовой ввод

- **Веб:** кнопка микрофона записывает звук в браузере (MediaRecorder; требуется HTTPS) и отправляет на сервер; расшифровка добавляется в поле вопроса — пользователь может её отредактировать.
- **Telegram:** голосовые, аудиофайлы и видеосообщения скачиваются, расшифровываются, бот показывает «🎤 Распознано: …» и открывает карточку нового вопроса (как для текста).
- **Обработка:** звук любого формата (OGG/Opus, WebM, MP4/M4A, MP3, WAV) декодируется библиотекой PyAV в WAV 16 кГц моно; максимальная длина — 5 минут. WAV отправляется модели `transcribe_model` через OpenRouter как аудио-вход с инструкцией дословно расшифровать речь. Стоимость расшифровки небольшая и учитывается отдельно от заседания.

11. Предварительная оценка стоимости

До запуска (CLI, веб, карточка в Telegram) показывается диапазон стоимости. Алгоритм (`estimate.py`):

1. Строится план вызовов каждого участника по режиму и раундам (как в таблице раздела 5), с учётом числа вариантов, которые он будет читать (n − 1 для своего).
2. Для каждого типа вызова используется эмпирическая оценка токенов: база + доля на каждый вариант для входа и выхода. Например, критика: вход 650 + 700·n, выход 260·n; итог: вход 900 + 850·n, выход 1500.
3. Добавляется объём вложений: токены документов (символы / 3) и ~1100 токенов на картинку для «зрячих» или ~350 токенов описания для «незрячих»; плюс вызов описания картинок.
4. Токены умножаются на цены моделей из каталога OpenRouter. Нижняя граница — это сумма; верхняя — ×2,5 (запас на рассуждения, повторы и длинные ответы). Для моделей без цены в каталоге стоимость не определена.

Также проверяется, что все модели из настроек существуют в OpenRouter; для неизвестных предлагаются похожие названия.

12. Хранение: архив, стенограмма, отчёты

Файлы в папке `runs/`

Файл	Содержимое
`<id>.json`	Источник правды: полная стенограмма (см. ниже). id = дата-время + краткое название вопроса.
`<id>.pdf / .docx / .md`	Полный отчёт.
`<id>.summary.pdf / .docx / .md`	Итог: решение, почему, особые мнения, риски.
`<id>.decision.pdf / .docx / .md`	Только решение совета: вопрос и итоговый ответ.
`<id>.files/`	Исходные файлы вложений.
`.reports-version`	Версия формата отчётов; при её изменении после обновления все отчёты пересобираются в фоне.

Содержимое стенограммы

Вопрос и уточнённый вопрос; критерии; режим; соответствие меток авторам; исходные и финальные предложения; отзывы последнего раунда критики; бюллетени; таблица голосования; итоговое решение; статистика по участникам (вызовы, токены, стоимость, сбои); общая стоимость; время начала и длительность; описания вложений и картинок; ссылка на родителя и цепочка уточнений; исходные настройки запуска; **журнал всех обращений** (этап, участник, фактическая модель, номер попытки, сырой ответ, разобранный объект или ошибка, стоимость, токены).

Отчёты

- Все форматы строятся из одного представления (`View`), поэтому их можно пересобрать в любой момент, в том числе для старых заседаний.
- Полный отчёт: заголовок (короткое название), исходный вопрос с сохранением форматирования, вложения, итоговое решение, таблица голосования, все предложения с авторами, критика, бюллетени, расходы, сбои.
- PDF: оглавление с номерами страниц и закладки; широкие таблицы ужимаются по ширине страницы. Word: поле оглавления (обновляется при открытии). Markdown: текстовая версия.
- Фирменное оформление: название, логотип, цвет и текст колонтитула из настроек.
- Безопасность генерации: в Markdown модели отключён «сырой» HTML; генератор PDF не загружает внешние ресурсы (только встроенные data-URL).

13. Интерфейсы: CLI, веб, Telegram

Командная строка (`python -m council`)

Команда	Назначение
`ask "вопрос"`	Заседание. Ключи: режим, раунды, критерии, варианты (`--option`, `--options-file`), контекст, вложения, `--followup`, `--mock` (демо без затрат), оценка стоимости с подтверждением. Живая строка прогресса с таймерами.
`models [поиск]`	Поиск моделей в каталоге OpenRouter.
`archive`, `reports`	Список заседаний; (пере)сборка отчётов.
`stats`	Статистика.
`web`, `telegram`	Запуск веб-сервера или бота.

Веб-интерфейс

- Одностраничное приложение (без фреймворков). Форма: вопрос, режим, раунды, участники, роли, бюджет, критерии, варианты, вложения, голосовой ввод; предварительная оценка стоимости.
- Заседание запускается как фоновая задача сервера; ход передаётся через Server-Sent Events с возможностью переподключения без потери событий (`Last-Event-ID`) — можно закрыть вкладку и вернуться.
- Живые карточки участников по этапам, полноэкранный просмотр любого ответа, итог, таблица голосования, отчёты, архив с поиском и фильтром, «Уточнить», «Повторить», статистика, уведомление об окончании.
- Основные адреса API: `POST /api/sessions`, `GET /api/sessions/{id}/events`, `POST …/cancel`, `GET …/report`, `POST …/repeat`, `POST /api/estimate`, `POST /api/uploads`, `POST /api/transcribe`, `GET /api/stats`.

Telegram-бот

- Доступ только для пользователей из `TELEGRAM_ALLOWED_USERS`; остальным бот сообщает их Telegram ID.
- Вопрос → карточка с настройками и оценкой стоимости → «Начать» → одно обновляемое сообщение с прогрессом и кнопкой «Остановить».
- Результат: короткий итог (до ~700 символов по границе абзаца) и кнопки: подробнее, голосование, полный отчёт, итог, только решение, «Уточнить», «Повторить».
- Команды: /settings, /budget, /evaluate, /archive, /stats, /stop, /cancel, /clear, /ask (в группах). Файлы и голосовые принимаются как вложения или вопрос. Одно заседание на чат одновременно; события одного чата обрабатываются по порядку.
- Демо-режим (без затрат) можно включить в настройках чата.

14. Статистика

Считается по всем сохранённым стенограммам (`stats.py`):

- число заседаний; общие расходы; расходы и число заседаний за текущий месяц; средняя стоимость и длительность; число заседаний с файлами и уточнений; распределение по режимам; помесячно за 12 месяцев;
- по каждой модели: участие, число предложений, побед (вариант модели выбран председателем), доля побед, средняя оценка критики, средний балл Борда, число сбоев, расходы всего и на заседание.

Модели сортируются по доле побед. Важно: статистика описательная; небольшое число заседаний, разные составы и вопросы не позволяют делать строгих выводов о качестве моделей.

15. Безопасность и развёртывание

- **Ключи** (`OPENROUTER_API_KEY`, токен бота, пароль веба) хранятся в файле `.env` на сервере и не попадают в код и архив.
- **Веб-доступ:** вход по логину и паролю (HTTP Basic, сравнение за постоянное время), блокировка IP на 15 минут после 10 неудачных попыток. Без пароля интерфейс должен работать только на localhost.
- **HTTPS:** обратный прокси Caddy с автоматическим сертификатом Let's Encrypt (для сервера без домена — домен вида `<ip>.sslip.io`).
- **Файлы:** проверка имён и расширений, ограничение размеров, безопасные имена при сохранении; идентификаторы заседаний проверяются по шаблону (защита от выхода за пределы папки).
- **Развёртывание:** Docker и docker compose (профили: бот, веб, HTTPS); установщик `install.sh` задаёт все вопросы с пошаговыми инструкциями и создаёт команду `ai-council` (status, logs, restart, update, ask, archive, reports, stats, config, password, access, backup и др.).
- **Данные** пользователей (вопросы, файлы) передаются в OpenRouter и далее провайдерам моделей — это стоит учитывать для конфиденциальной информации.

16. Известные ограничения и вопросы для анализа

Ниже — места, где логика опирается на допущения. Их полезно оценить при анализе.

Методология совета

1. Анонимность неполная

Модели могут узнать «свой» или чужой стиль и формат. Метки скрывают имя, но не манеру письма. Возможен эффект предпочтения вариантов, похожих на собственный.

2. Председатель — тоже участник

Председатель пишет итог, будучи автором одного из вариантов (ему известна только своя метка косвенно — по содержанию). Нейтральность обеспечивается только инструкцией. Он может выбрать победителя вопреки голосованию (с обоснованием). Альтернатива: председатель без собственного варианта или итог строго по голосованию.

3. Средняя оценка критики относится к версиям до последней доработки

В режиме «Дебаты» критика оценивает варианты до доработки, а голосование — после. В таблице итогов средняя оценка критики служит вторичным критерием, но описывает предыдущую версию варианта.

4. Равный вес критериев и голосов

Все критерии и все бюллетени весят одинаково; уверенность модели (`confidence`) в подсчёте не используется. Число голосующих за вариант может различаться (выбывшие участники), нормировка Борда это частично компенсирует.

5. Конформизм при доработке

Модели видят критику и чужие варианты и могут сходиться к одному решению. Это повышает согласованность, но может снижать разнообразие. Инструкции требуют «не менять в угоду критикам», но гарантий нет.

6. Уточнённая формулировка задаётся одной моделью

Председатель на этапе 0 переформулирует вопрос и выбирает критерии; ошибка на этом шаге влияет на всё заседание. Пользователь может задать критерии сам.

Технические аспекты

7. Бюджет может быть немного превышен

Проверка лимита выполняется перед каждым запросом, но запросы одного этапа уходят параллельно — превышение возможно на стоимость уже отправленных запросов.

8. Голосование без вложений

На этапе голосования модели не получают документы и картинки (только тексты вариантов). Это экономит деньги, но ранжирование может не учитывать детали материалов.

9. Контекст уточнений ограничен

В уточнение передаются только итоговые решения (до 4000 символов) последних 4 заседаний цепочки, без хода обсуждения.

10. Воспроизводимость

Результаты не детерминированы (температура 0,7, разные версии моделей у провайдера). Повтор заседания — новое независимое обсуждение. Фактическая модель, ответившая на каждый запрос, записывается в стенограмму.

11. Оценка стоимости приблизительная

Основана на эмпирических объёмах токенов; верхняя граница с запасом ×2,5. Фактическая стоимость берётся из ответов OpenRouter.

12. Фактическая проверка

Модели не проверяют факты по интернету (кроме участника perplexity, выключенного по умолчанию). Согласие нескольких моделей не гарантирует фактической верности.

Возможные направления развития

- Учёт уверенности моделей и весов критериев в подсчёте; доверительный интервал по бюллетеням.
- Отдельный «нейтральный» председатель без своего варианта; вариант итога строго по голосованию.
- Повторная критика финальных версий перед голосованием (или использование оценок финальных версий).
- Метрики согласия совета (например, коэффициент согласованности ранжирований) как сигнал надёжности ответа.
- Инструменты проверки фактов (поиск) для одного или нескольких участников.

Приложение. Карта исходного кода

Файл	Содержимое
`council/session.py`	Класс `Council`: этапы 0–6, `_ask` (запрос + проверка + повторы), `_parallel`, бюджет, кворум, события, `SessionResult`.
`council/prompts.py`	Системная инструкция и шаблоны запросов всех этапов, блок истории для уточнений, описание картинок, просьба исправить JSON.
`council/schemas.py`	Схемы ответов моделей.
`council/voting.py`	Извлечение JSON, метки, нормализация критики и ранжирования, подсчёт Борда.
`council/client.py`	Клиент OpenRouter: поток, повторы, пустые ответы, учёт стоимости, каталог моделей.
`council/config.py`	Настройки, проверка, переопределения, чтение `.env`.
`council/estimate.py`	План вызовов, оценка стоимости, проверка моделей, определение «зрения».
`council/attachments.py`	Чтение файлов, извлечение текста, уменьшение картинок, лимиты, объединение.
`council/transcribe.py`	Декодирование аудио и расшифровка.
`council/archive.py`, `report.py`	Архив, сохранение, поиск, отчёты, цепочки уточнений, исходные настройки.
`council/stats.py`	Статистика.
`council/reports/`	`view.py` (представление), `html.py`/`pdf.py`, `docx.py`, `md.py`, `brand.py`.
`council/cli.py`	Командная строка.
`council/web/`	`app.py` (API, SSE), `auth.py` (пароль), `static/` (страница).
`council/telegram/`	`bot.py` (логика бота), `api.py` (Bot API), `format.py`.
`council/mock.py`	Демо-клиент без затрат (для тестов и пробы).
`config.yaml`, `install.sh`, `Dockerfile`, `docker-compose.yml`	Настройки, установщик, развёртывание.
`tests/`	Автотесты (78): оркестратор, клиент, оценка, вложения, отчёты, веб, бот, новые функции.